NetBox ISC DHCP – manual

Plugin for NetBox 4.7+ that builds the ISC DHCP server configuration from NetBox data. Manual version: v0.17.1

1. What the plugin does

From NetBox objects (tenants, prefixes, DNS servers) the plugin builds ISC DHCP files: the main `dhcpd.conf`, class and static host tables for tenant groups, and subnet files per pool type. Files are stored ready-made (with a SHA256 hash) and recomputed when the data changes. The DHCP server downloads them with its own script.

2. Installation and update

```
sudo bash -c 'source /opt/netbox/venv/bin/activate && pip install --upgrade --force-reinstall --no-deps "git+https://github.com/ArK761/NetBox_ISC-DHCP.git@<COMMIT>" && cd /opt/netbox/netbox && python manage.py migrate netbox_isc_dhcp && systemctl restart netbox netbox-rq'
```

Enable the plugin in `configuration.py`: `PLUGINS = [..., "netbox_isc_dhcp"]`. Version and commit are shown top right on every plugin page.

3. Menu

Item	Content
Plugin CFG	Language (Slovak / English), path of `dhcpd.conf`, base path of files, subdirectories.
DHCP-CFG	Hub: Base dhcpd.conf, DNS servers, Subnets, Hosts and MAC addresses, Own variables.

Values are edited by clicking on them (a field and a Save button appear).

4. Plugin settings (Plugin CFG)

- **dhcpd.conf path** – directory on the DHCP server, e.g. `/etc/dhcp`.
- **Base path of files** – e.g. `/opt/dhcp`; subdirectories live below it.
- **Subdirectories** – e.g. `TABLE`, `SUBNET`; giving `/opt/dhcp/TABLE`.
- **Default subdirectory for tables** and **subdirectory for subnets**.

Everything is empty by default – the user sets it.

5. Base dhcpd.conf

- **Static part** – global DHCP server settings (authoritative, lease times, logging, primary subnet). **It must be written by the user**; the plugin never changes it.
- **Dynamic part** – the plugin writes **only INCLUDE lines** into it (paths of table and subnet files), never configuration.
- The **Preview** tab shows the whole resulting file.

6. DNS servers

Table Name + IP + **In use** (checkbox). At most 2 can be in use at once. If none is in use, the plugin shows a warning. DNS servers in use are available in pool code as `{{ dns }}`. All DNS servers (also unused) can be used in Own variables.

7. Subnets and pools

Subnet source

1. Choose the **object** (e.g. IPAM › Prefix) and the **subdirectory for subnets**.
2. **Add field** – any field of the object (standard or custom field).
3. Click a row – the list of all values opens with search, **Select all** / **Untick all** and the switch **only ticked** / **all except ticked**. Nothing ticked = *any (no filter)*.
4. On the field that defines the pool type (e.g. `POOL_TYPE`) click the **key** icon. A field with more than 100 values cannot be chosen.

Below the conditions you see the number of selected objects and the **gateway (GW)** for all prefixes: first or last usable IP.

Pools and their code

Every pool type has its own **code (template)**, a **Generate** switch and a **preview** on a real prefix. Variables in the code:

Variable	Meaning
`{{ network }}`, `{{ netmask }}`, `{{ prefix }}`	network address, mask, prefix (e.g. 10.0.0.0/24)
`{{ gateway }}`	gateway by the global setting
`{{ first_usable }}`, `{{ last_usable }}`	first / last usable address (never network or broadcast)
`{{ ip(n) }}`	n>0 from the network address, n<0 from the broadcast (−1 = last usable)
`{{ dns|join(",") }}`	DNS servers in use
`{{ cf.FIELD }}`, `{{ obj }}`	custom fields and the object itself

Example:

```
subnet {{ network }} netmask {{ netmask }} {
    USE LEASE_600
    option routers {{ gateway }};
    option subnet-mask {{ netmask }};
    pool {
        range {{ ip(2) }} {{ last_usable }};
    }
}
```

> If the code does not fit a prefix (e.g. ranges for /24 used on /28), the preview shows a red error and such a file is not used.

8. Own variables

Named pieces of ISC configuration. When adding one, pick an ISC option (lease, DNS, network, access, PXE, DDNS, own text), write a description and set the value right away:

Option	Value	Inserts
default + max lease	600	`default-lease-time 600;` `max-lease-time 780;` (factor 1.3 – configurable)
domain-name-servers	ticked DNS servers	`option domain-name-servers …;`
range	10.17.0.10 10.17.3.254	`range 10.17.0.10 10.17.3.254;`
allow / deny duplicates	deny	`deny duplicates;`

In pool code type `..` – a list of variables appears at the cursor; pick with the arrows and Enter. A line `USE <name>` is inserted and replaced by the real lines in the result. When inserted with `..`, the comment `## VARIABLE ADDED VIA OWN VARIABLES` is added above the line. Variable lines are yellow in the editor and in the preview, and in the generated file every line coming from a variable ends with `## added via own variable <name> ##`. A variable in use cannot be deleted; when renamed, the new name is written automatically into every code (pools, group files and individual subnet codes). If a code contains `USE` with a name that has no variable (e.g. renamed in an older version), a red bar is shown at the top and the Own variables page has a card “Code uses variables that do not exist”: pick an existing variable and click **Rewrite** – the lines are rewritten in every code.

9. Hosts and MAC addresses

Page **DHCP-CFG › Hosts and MAC addresses** (work in progress – grows step by step). Everything is set by clicking:

1. **Table location** – the **table subdirectory** (full path shown, e.g. `/opt/dhcp/TABLE/`).
2. **Table source (groups)** – a tree: the object whose every item (e.g. Tenant group) gets two files `<Name>_Dynamic` and `<Name>_Static`, and the **name** field (built-in or custom field, usually Name). Turn off a group you do not want with the switch in the groups list.
3. **Groups (tables)** – a list like the pools: **On / Off** switch, number of objects, files with full path, sources, MAC counts and warnings. A group switched **off** still has its files, but without MAC addresses – base configuration only, so DHCP does not fail. **Set up** opens the group page.
4. **Group page** – at the top the summary **"N hosts will not get into DHCP"** (no MAC, no IP, invalid MAC/IP, name not allowed, same host name) and the warnings; both expand on click. The group is turned on / off only in the groups list.
    - **Host sources** (**Add source**): object (e.g. Tenant, Device), **conditions** on an internal field or custom field (which objects; like on Subnets) and the host name field.
    - **Hosts of one object** (at most 3): every row = **MAC field** + **IP field** (optional). A row **without an IP field** always goes to `_Dynamic` (class match). A row **with an IP field** goes to `_Static` (host block); the **Also into class** switch (shown only with an IP field) puts the same MAC into `_Dynamic` too, so the device gets an IP in another pool (otherwise MismatchClient). If the IP field is empty (or invalid), the host cannot be static: with “Also into class” it goes to `_Dynamic` only, without it it does not get into DHCP (summary “no IP”). The same MAC field may be in two rows only as one static row (without “Also into class”) + one row without IP (Dynamic); twice in Static or twice in Dynamic is refused with an explanation (duplicate hosts). Every field can be used once. Column **Goes to**: Dynamic / Static / Static + Dynamic.
    - Names: one row = `NAME`, more rows = `NAME_1`, `NAME_2` … by row order. Two host blocks with the same name are never made; a duplicate MAC or IP is only a warning.
    - **option host-name** (static row): **NAME** (default), **NAME_n** (as the host block name), **another field** of the object or **leave out**.
    - **Files**: path, suffix (`_Dynamic` / `_Static`, editable), host count and **Show CFG** = preview of the finished file from real data; the MAC/IP list expands on click.

    Copy settings copies the sources of another group.

File code – every group has its own code (like a pool), on the group page at each file (**Code**, expands on click); the groups list marks edited code. Jinja code for `_Dynamic` (variables `class_name`, `pairs`, `tenants`) and `_Static` (`class_name`, `hosts`: `h.host`, `h.mac`, `h.ip`, `h.name`, `h.hostname`). State default / edited, **Restore default**. Bad code is not saved (the draft stays in the editor). A group switched off has an empty class (`match if false;`) and no host blocks.

Next step: storing the files (SHA256) and including them in dhcpd.conf.

10. Troubleshooting

Symptom	Cause / fix
Red DNS warning	Switch on at least one DNS server.
Pools have number names	The key is on the wrong field – put it on the pool type field.
"Selected: 0 objects"	Check the conditions; nothing ticked means the field does not filter.
Error in pool preview	The code cannot be used for that prefix – adjust ranges (use `ip(n)`, `last_usable`).

11. Pool code check

On every save the plugin tries the code on **all prefixes of the pool**. If it finds an error, the code is **not saved** (the last working code stays), the errors are listed with the number of prefixes and an example, and your text stays in the editor. Checked:

- everything is inside one `subnet <network> netmask <mask> { }` block, network and mask match the prefix,
- `;` at the end of statements, closed quotes, parentheses and `{ }`,
- unknown or misspelled statements and options (standard ISC options + own options from the static part `option X code N`),
- the same option twice in one block,
- values: IP addresses (routers, DNS, NTP…), numbers (lease times), quoted text (domain-name, filename…),
- `option routers` on subnet level must be a usable address, `option subnet-mask` must match the mask, `default-lease-time` must not exceed `max-lease-time`,
- classes in `allow/deny members of "X"` must exist (tenant tables or static part),
- words in `log (...)` expressions – e.g. `foo` must be defined with `set foo = …` in the static part,
- `range` inside the network, not overlapping, gateway outside ranges, `ip(n)` within usable addresses, unknown `USE`.

> The check does not replace `dhcpd -t` on the DHCP server, but it catches the common errors that would stop the DHCP server from starting.

12. Pool sizes and incompatible subnets

- **Code is suitable for** – on save the plugin tries the code on sizes /16 to /30 and shows which work (e.g. /16 – /27).
- **Use for** – narrow where the code is used (e.g. only /24). It cannot exceed the suitable range; "auto" = the whole suitable range.
- A subnet whose size is outside the range (e.g. changed from /24 to /28 in NetBox) or that the code does not fit is **incompatible**: it is not exported and the DHCP server

keeps running without it.
- A red bar is then shown at the top of every plugin page; the list with reasons is on the Subnets page and on the pool, and the pools table shows a red count.
- Incompatible subnets do not block saving the code – only subnets within the range are checked.

13. Individual change of a subnet

- On the pool, switch on **Allow individual changes of subnets**.
- At an incompatible subnet (or in the preview for the chosen prefix) click **Individual change**. The editor opens with the pool code as a start; you change it for this subnet only.
- On save the same check applies, but only to this subnet. The pool's size range is not checked for a subnet with its own code.
- Changes of the pool code do not affect individual subnets – on save the plugin lists which subnets are not affected.
- A yellow bar at the top of the plugin pages shows the number of subnets with an individual configuration; the list (pool, subnet, state, changed) is on the Subnets page and on the pool.
- The **Code** column in the pools table (and on the pool page) shows whether the pool uses the plugin **default** code or already **edited** code, and the number of subnets with an individual change (**+N individual**). From the pool page, **Back to Subnets** or the breadcrumb at the top (DHCP-CFG / Subnets) returns to Subnets.
- The **Show tree ▾** button in the bar expands the tree pool → subnets; clicking an item opens it in a new window.
- If the pool does not allow individual changes or the subnet is no longer in the pool, the individual code is not used (the state says so).
- **Remove individual change** returns the subnet to the pool code.